VIDEO_SCRIPT 提词版

演示视频讲稿(12 分钟标准版 / 25 分钟展开版)

配套脚本:`scripts/demo_sp3.sh`(八步时间线,实时约 5–6 分钟)。队伍:西安邮电大学 · 三驱兄弟 · T2026116649912743 · 选题 proj2《面向 openEuler 的自适应资源管控 Agent》。

> 提词版(每镜一页、大字)由 `python scripts/build_teleprompter.py` 生成为 `docs/EulerPilot-v2演示提词版-v1.1.pdf`;本文件仍是唯一事实源,改稿后重跑即可。

口播原则:只念屏幕上真实出现的行;所有性能数字用彩排实测值并标注"预注册协议下的配对结果";做不到的事(跨机、adaptive、上游 scx 对照)主动说 UNSUPPORTED, 不回避。带【现场值】标记的数字以录制当次输出为准,不要照念稿子里的数。

0. 录制前 30 分钟检查单

项	命令 / 动作	期望
机器空闲	`cat /sys/kernel/sched_ext/state`	`disabled`
无残留 pin	`ls /sys/fs/bpf/eulerpilot-v2`	空或不存在
无并发写者	确认 `schedpilotd` 未运行(与本项目互斥)	未运行
无他人任务	`flock -n /run/lock/eulerpilot-v2-acceptance.lock true && echo free`	`free`
内核与产物	`uname -r;bash scripts/verify_package_sp3.sh <BUILD> <SRC>`	`6.6.0-schedpilot-sp3;all checks passed`
先跑一次彩排	完整跑一遍 `demo_sp3.sh`	末行 `[demo] PASS`,收尾 `pins=0`
提示符脱敏	把 `PS1` 改成 `[root@demo ~]#` 一类,或在 SP3 控制台/VNC 直接录	画面不出现内网地址、跳板机名、密钥文件名
画面准备	终端字号 ≥ 16px、窗口 1280×800 起、左窗格 demo、右窗格只读命令	手机也能看清
素材准备	打开性能表(`docs/REPORT.md` §6 表)与 fail-safe 结果 `results.tsv`	镜 7 用

> demo 第 4、7 步要等真实采样窗口与饥饿形成(脚本内有 sleep),这两段是**天然留白**, 口播按稿子填满即可,不要快进——评委要看的就是”它真的在跑”。

1. 分镜与时间码总表

镜	时间码	画面	操作	对应 demo 步骤
0	00:00–01:00	标题页 + 三层架构图	无	—
1	01:00–03:00	左窗格 demo 输出	跑 `demo_sp3.sh`,讲执行服务与契约	第 1 步
2	03:00–04:00	基线输出	讲默认调度器基线	第 2 步
3	04:00–05:30	`VERIFIED` 与内核自证行	讲挂载与读回	第 3 步
4	05:30–07:30	感知链三行输出	右窗格补敲 `ep features/route/slo`	第 4 步
5	07:30–09:00	SIGKILL 与重启	讲崩溃恢复与 fail-safe 矩阵	第 5 步
6	09:00–10:00	`cpu.weight` 读回	讲第二效果域	第 6 步
7	10:00–12:30	事件链升权与补偿	讲 R5 全链路	第 7 步
8	12:30–13:30	收尾断言行	讲”零残留”	第 8 步
9	13:30–15:30	性能表 + fail-safe 表	读表与边界	—
10	15:30–16:00	结束页	一句话收束	—

> 标准版剪到 12–13 分钟:镜 9 压到 1 分钟(只念主表三行 + 边界一句),镜 2/6 各砍半。

2. 逐字口播

镜 0 • 开场(1 分钟,配架构图)

> 各位老师好,我们是西安邮电大学”三驱兄弟”队,选题 proj2《面向 openEuler 的自适应资源管控 Agent》。先用一句话说清这个系统:**用户态 Agent 负责感知和决策,特权执行服务是唯一的写者,内核里由我们自己写的 sched_ext 调度器执行。**每一条策略都是声明式契约,每一次生效都走同一个事务——快照、写前日志、生效、读回验证、提交;任何一步失败, 系统一律向”没有策略”这个安全方向回退。
>
> 接下来这段是**一次不间断的真机演示**,跑在 openEuler 24.03-LTS-SP3 上,没有回放、没有 mock。八步走完,我会用两张表说明性能与故障恢复的证据从哪来。

镜 1 • 唯一写者与契约(demo 第 1 步)

> 启动的是执行服务 `ep-executord`。它独占一个状态目录——flock 单实例、租约加栅栏、世代号、只追加的写前日志。屏幕上这行 `-- instance=… skill=sched generation=… degraded=false`,就是它的自述。
>
> 现在这份策略是一个 JSON 契约:它声明目标是哪个 cgroup、业务角色是前台还是后台、用哪个专家档位、参数边界,以及安全约束——最短驻留、观测最长可容忍年龄、策略 TTL。`ep validate` 先做一遍 schema 加语义校验,注意这一行 `-- policy validated`,**校验不过的策略根本进不了事务**,更碰不到内核。

镜 2 · 工作负载与基线(demo 第 2 步)

> 被测负载是 Redis,放在一个 systemd scope 里,这样它有明确的 cgroup 身份。先在**默认内核调度器**下测 8 秒基线,屏幕上是【现场值】。这一臂就是我们所有实验里的 A0——不加任何策略,同一台机器、同一个内核启动。

镜 3 • 生效并由内核自证(demo 第 3 步)

> 提交生效。返回 `-- VERIFIED, generation N` 只是执行服务的说法,真正作证的是内核: 这两行 `sched_ext=enabled`、`ops=eulerpilot`,以及**从内核 map 里读回来的分类** `-- declared class read back from the kernel map: LAT`。
>
> 这里有个设计值得说:挂载调度器的 loader 是个一次性进程,注册完成后把 link 文件描述符通过 SCM_RIGHTS 交给执行服务持有,自己就退出了。所以**任何时刻内核里只有一个写者**,而且执行服务一旦死亡,文件描述符关闭,内核立刻把调度器摘掉。

镜 4 • 感知与决策链(demo 第 4 步)

> 现在制造 CPU 扰动:四个 renice 过的自旋任务。感知链逐级应答,我一条一条敲。
>
> 第一条 `ep features`——特征窗报 CPU 忙度、负载和 PSI,每个指标都带有效性和年龄, 采不到就标 invalid,不会拿旧值冒充。第二条 `ep route`——路由器给出建议档位和理由,注意它**只是建议**。第三条 `ep slo`——门给结论:`allowed=… code=…`。
>
> 关键设计在这里:观测只能让门**更严**,不能让门放行。脚本喂进一条实测 p99 6.4 毫秒的服务观测,再看同一个门——`FOREGROUND_P99_EXCEEDED`,拒绝切换。还有一条护栏是踩过坑才写进协议的:我们声明的比较敏感度是 5%,但实测同一状态下 p99 的**窗间噪声极差达到 46%**。敏感度低于噪声的判定,我们明报 `not run`, 而不是给一个看起来精确的数字。

镜 5 • 崩溃恢复(demo 第 5 步)

> 现在把执行服务 `kill -9`。看这一行:`after SIGKILL: sched_ext=disabled` —— **不需要任何清理代码**,link fd 一关,内核就把调度器摘了。重启后:`degraded=false`、孤儿 pin 被同实例回收、世代号延续,不是从头计。
>
> 这个方向感我们叫它"失效方向不对称":**拆除效应永远放行,建立新效应必须有可持久化的审计**。所以即使状态目录所在的磁盘被写满,系统仍然能回退。这一类行为不是口头承诺, 是 17 个故障场景的真机矩阵:进程崩溃、loader 崩溃、租约超时、策略 TTL、非法与越界策略、世代号冲突、双实例、外部越权改值、成员迁出、磁盘写满、CPU 热插拔、两步写入的第二步失败……每个场景都断言三件事:业务进程继续推进、内核状态由 map 自证清除、没有新增残留。

镜 6 • 第二个效果域:cgroup(demo 第 6 步)

> 赛题要求能扩展到别的资源域。这里是 cgroup 域:策略声明 `cpu.weight` 调到 300, 屏幕这行 `cpu.weight now 300 (was 100, read back from the kernel file)` 是**写后读回**的结果,原值 100 记在 marker 文件里,进程崩溃后重启会自动恢复。值得强调的是:加这个域**没有改动核心事务代码**,策略的 kind 和实例的域不匹配时, 直接零副作用拒绝。

镜 7 • R5 事件链:hook → 决策 → 动作 → 恢复(demo 第 7 步)

> 这是扩展要求的完整链路,不只是一个 hook。左边,eBPF 程序挂在 raw tracepoint 上, 测每个 cgroup 里任务从唤醒到首次运行的等待;一个**只读泵**把聚合结果排空上报, 这段演示里窗口 3 秒、每 0.5 秒上报一次,它自身开销实测 0.5% 单核、约 12k 事件每秒、丢弃计数为 0。右边,被人工授权的 `EventArm` 武装起来的执行服务。
>
> 现在让被看的 cgroup 饿着——等它的窗口。看这行:`cpu.weight` 从 100 升到 5000, **走的是和人工策略完全同一条事务路径**,不是旁路。撤掉负载,条件消失,自动补偿回原值。
>
> 还有两条兜底:杀掉泵,观测断供,门立即冻结,已生效的效应由 TTL 撤销;把同一个路径的 cgroup 删除重建,身份不匹配,授权失效并明报 `WATCH_REBUILT`——**权限永远不会静默转移给一个没人授权的新对象。**

镜 8 • 收尾断言(demo 第 8 步)

> 收尾不是”我们以为干净”,是断言出来的:`sched_ext=disabled`、 `pins=0`,并且这一行给出了本次演示的证据目录,里面有守护进程日志、写前日志和每一步的原始输出。这个脚本在交付前连续彩排三次,每次都是零残留。

镜 9 • 性能与归因(读表)

> 性能结论来自**预注册**的实验协议:开环负载、块内配对、一万次配对 bootstrap、固定随机种子,无效规则先写后跑。主表这一行:固定专家臂对默认调度器, 60k rps、SLO 5 毫秒下,goodput **+1.83%,置信区间 1.38 到 2.26,20 个实验块全胜**, p99 降 **39.6%**;后台任务相对进度 0.998 到 1.006,**前台的收益没有饿死后台**。
>
> 更重要的是我们把收益拆开归因:纯观测栈是净成本,约 −0.5%;BASIC 数据面 +1.75%; 对静态 cgroup 倾斜的额外收益 +1.90%;而 class 路由的配对消融**是零**。三条独立证据一致,所以我们的结论是:**class 声明是可审计的契约,不是性能开关**。
>
> 边界也一并报:收益集中在 5 毫秒以上的远尾,**2 毫秒阈值处没有优势甚至略负**; 单机、单负载、单 SLO;跨机验证是 UNSUPPORTED,因为备用机是 cgroup v1 且没有 PSI; 上游 scx 调度器对照也是 UNSUPPORTED,因为上游主线已 Rust

> 化、目标内核高于本机 6.6。参考实现 schedpilotd 在同一协议下 +1.92%,与我们的结果同量级。

镜 10 • 结束页(30 秒)

> 代码、契约、逐轮二进制哈希、失败轮现场和这份演示脚本都在仓库里,包校验脚本端到端全过,交付镜像 14.6GB 含可复现的构建配方。我们的主张不是”跑分高”,而是:**自适应能力本身几乎免费,而每一次生效都可验证、每一次失效都可恢复**。谢谢。

3. 25 分钟版追加块(每块 2–3 分钟,插在对应镜后)

插在哪	展开话题	可用的实测材料
镜 4 后	观测分辨率护栏的由来:静息 p99 1.7ms 主因是 idle 退出延迟;加载 30s 窗 p99 CV 20.2%	`docs/EXPERIMENT_PROTOCOL.md` §5.1
镜 5 后	两个由矩阵发现的真实产品缺陷:停止信号不清理 pin(新增 `shutdown()`)、盘满时回退被租约前置写挡死(方向不对称)	`docs/p1/P1_EXIT_REPORT.md` §5
镜 5 后	内核级根因两则:`scx_enable_seq` 在 state 可读之后才发布(曾致 7 次验收败 2);绑核任务被改投全局 DSQ 触发 runnable stall(修复前 6/6 失败 → 3×90s 零失速)	`evidence/p1/enable-seq-publish-20261022.txt`、`stall-runnable-20261023.txt`
镜 7 后	门控的真实作用是”延迟器”:同一注入下无门臂第 6 次轮询(约 12s)切换、有门臂第 26 次(约 52s),**推迟约 40 秒**——自动化不是越快越好	`evidence/p2/gate-injection-20261023.txt`
镜 9 后	切换代价为什么必须量化:挂载/卸载空窗实测 303–665ms,空窗内 1ms 探针 p99 由 1.2–2.0ms 升到 13.2–16.1ms	`docs/HANDOVER_20261024.md` §3.2
镜 9 后	溯源与脱敏纪律:逐轮哈希、三代冻结构建、公开镜像走独立脱敏历史线	`PROVENANCE.md`

4. 现场失败预案

情况	处置
某步 `[FAIL]`	不要剪辑掩盖。脚本会保留现场目录并打印路径;当场重跑一次,若仍失败,把这段保留并在视频里说明失败原因与定位过程——这本身就是 R7 负路径的展示材料
事件链没触发	多半是负载稳态贴着阈值;按 `docs/用户手册.md` 第 7 章调 `threshold_wait_ms` 或海绵进程优先级,重录镜 7
有调度器在位起不来	先 `cat /sys/kernel/sched_ext/state` 与 `ops`,由在位者属主回退;确认 `schedpilotd` 未运行
中途机器被占用	演示与实验统一用 flock 锁串行,不要并发;等锁释放再录
时间超预算	优先砍镜 2、镜 6 的重复解释,**不要砍镜 9 的边界声明**

5. 录制后动作

1. 上传后把链接填到两处: `README.md` 的”演示视频讲稿”行、`docs/REPORT.md` §8;
2. 若视频里的数字与稿子中的【现场值】不同,以视频为准并回填 `docs/REPORT.md` 说明 (性能主张仍以冻结矩阵结论为准,不用演示数据替代);
3. 归档本次 demo 的证据目录(含 `daemon.log`、`journal.jsonl`)到 `evidence/p5/`,并在 `SHA256SUMS` 里登记。

6. 修订记录

版本	日期	内容	作者
v1.0	2026-10-25	逐字讲稿(六节,15分钟)	三驱兄弟
v1.1	2026-10-26	重排为分镜制:录制检查单、时间码总表、逐字口播对齐 demo 八步屏幕输出、25 分钟版追加块、失败预案、录制后动作;数字与证据文件对齐	三驱兄弟